

Processamento de Consulta em Bancos de Dados



Eduardo Ogasawara

eogasawara@cefet-rj.br | <https://eic.cefet-rj.br/~eogasawara>



Etapas Básicas no Processamento da Consulta

O processamento de uma consulta em um banco de dados relacional passa por três etapas fundamentais que transformam uma requisição SQL em resultados concretos.

Análise e Tradução

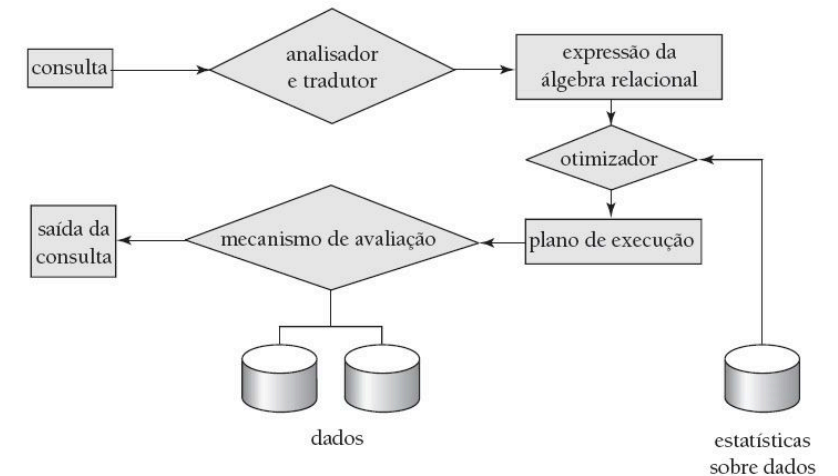
Converte a consulta SQL em uma representação interna, verificando sua validade sintática e semântica.

Otimização

Identifica o plano de execução mais eficiente entre múltiplas alternativas equivalentes.

Avaliação

Executa o plano escolhido e retorna os resultados ao usuário.



Análise, Tradução e Avaliação

Análise e Tradução

Transformações e verificações essenciais na consulta SQL:

- Traduzida para expressões de álgebra relacional
- Analisador sintático verifica a gramática da consulta
- Analisador semântico valida relações e atributos no esquema

Avaliação

Executa o plano selecionado e produz os resultados:

- Executa cada operação do plano na ordem especificada
- Acessa os dados necessários em disco ou memória
- Retorna as tuplas resultantes ao usuário

Etapas Básicas no Processamento da Consulta: Otimização

A otimização de consultas é o coração do processamento eficiente em SGBDs, determinando como uma consulta será executada e impactando diretamente no tempo de resposta e uso de recursos.

O otimizador analisa múltiplas estratégias de execução e, com base em estatísticas do catálogo, estima o custo de cada alternativa e seleciona a de melhor desempenho.

Árvore de Expressão como Plano de Avaliação

A árvore de expressão representa internamente uma consulta como estrutura hierárquica de operações da álgebra relacional.

Estrutura da Árvore

Nós internos representam operações (seleção, projeção, junção); folhas representam as relações base

Planos Equivalentes

Múltiplas árvores podem representar a mesma consulta com custos de execução distintos

Escolha Otimizada

O otimizador seleciona a árvore de menor custo estimado com base em estatísticas e heurísticas

Processo de Otimização de Consultas

Análise de Alternativas

Entre todos os planos de avaliação equivalentes, o otimizador identifica e compara múltiplas estratégias de execução

Estimativa de Custo

Utiliza informações estatísticas do catálogo (número de tuplas, tamanho de registros, distribuição de valores) para calcular o custo esperado

Seleção do Melhor

Escolhe o plano com menor custo estimado para execução real

Objetivos do Processamento de Consultas

- Desenvolver métricas precisas para medir custos de diferentes estratégias de execução
- Implementar algoritmos eficientes para avaliar operações da álgebra relacional
- Combinar operações em planos de execução completos e otimizados

No módulo de otimização de consultas, o foco está em encontrar o plano de avaliação com menor custo estimado. Técnicas como transformações algébricas e seleção de métodos de acesso são aplicadas para esse fim.

Medidas de Custo da Consulta

O custo de execução de uma consulta é tipicamente medido pelo tempo total necessário para produzir os resultados, sendo o acesso ao disco o principal fator.

Componentes do Custo de E/S em Disco

Operações de Busca

Número de buscas × custo médio de busca — O tempo para posicionar o cabeçote de leitura na posição correta

Leitura de Blocos

Número de blocos lidos × custo médio de leitura — A transferência de dados do disco para a memória

Escrita de Blocos

Número de blocos escritos × custo médio de escrita — Operações de escrita são mais custosas que leituras devido a verificações adicionais



Simplificações na Modelagem

Para facilitar a análise, frequentemente utilizamos o **número de transferências de bloco** como medida única de custo.

Podemos ignorar:

- Diferenças entre E/S sequencial e aleatória
- Custos de processamento da CPU
- Custos de comunicação em rede

Influência da Memória nas Medidas de Custo

A quantidade de memória disponível para buffers afeta dramaticamente o custo real de execução das consultas.

Desafio da Previsão

A quantidade real de memória disponível é difícil de determinar antecipadamente, pois depende de processos concorrentes e outras operações do SGBD.

Estratégia Conservadora

As estimativas de custo assumem o **pior caso possível**, supondo que apenas a quantidade mínima de memória necessária estará disponível.

Impacto no Desempenho

Mais memória pode transformar operações com múltiplas passadas pelo disco em operações realizadas completamente em memória, reduzindo drasticamente o tempo de execução.

Dominando as Operações da Álgebra Relacional

Cada operação fundamental da álgebra relacional pode ser implementada de múltiplas formas, cada uma adequada a diferentes cenários.

Seleção

Filtra tuplas que satisfazem um predicado, usando varredura completa ou índices primários, secundários e combinados.

Ordenação

Organiza tuplas por atributos, com Quick Sort para dados em memória e Merge Sort externo para grandes volumes.

Junção

Combina tuplas de duas relações via Loops Aninhados, Hash Join ou Merge Join, cada um ideal para contextos distintos.

Avaliação da Operação de Projeção

A projeção elimina atributos de uma relação, mantendo apenas as colunas especificadas. O principal desafio é a **eliminação de duplicatas**, necessária quando os atributos projetados não formam uma chave.

Projeção Baseada em Ordenação

Ordena as tuplas projetadas e remove duplicatas consecutivas em uma única passada.

O custo depende do tamanho da relação, da necessidade de eliminar duplicatas e da memória disponível.

Projeção Baseada em Hashing

Usa uma função hash para particionar as tuplas e eliminar duplicatas dentro de cada partição.

Varredura Completa: Seleção por Varredura de Arquivo

Varredura Linear Completa

O método mais simples: examina cada bloco do arquivo sequencialmente, testando o predicado de seleção em cada tupla

Custo de E/S

Requer br transferências de bloco, onde br é o número de blocos que a relação ocupa no disco

Quando Usar

Adequado para predicados de baixa seletividade ou quando não há índices disponíveis sobre os atributos do predicado

A varredura de arquivo é o método de acesso mais básico e funciona para qualquer predicado de seleção. É eficiente quando:

- O predicado tem **baixa seletividade** (seleciona grande porcentagem das tuplas)
- Os dados estão armazenados de forma **sequencial** no disco
- O sistema operacional pode realizar **leitura antecipada** (read-ahead)

Não requer estruturas auxiliares ou índices, mas sempre examina toda a relação, mesmo para predicados altamente seletivos.

Acesso Indexado: Seleção via Índices

Índices são estruturas auxiliares que permitem acesso rápido a tuplas específicas sem examinar toda a relação.

Índice Primário

Construído sobre a chave de ordenação do arquivo, permite busca binária eficiente e acesso sequencial rápido.

Índice Secundário

Construído sobre atributos diferentes da chave de ordenação do arquivo (podem ser chaves candidatas ou não); pode apontar para múltiplas tuplas espalhadas pelo disco e geralmente é mais custoso que o índice primário.

Índice Hash

Utiliza função hash para localizar rapidamente tuplas, ideal para buscas por igualdade, mas não suporta consultas de intervalo.

Escolha entre Varredura e Uso de Índices

O otimizador deve considerar múltiplos fatores para decidir entre usar um índice ou varredura sequencial.

Seletividade do Predicado

Quantas tuplas satisfazem a condição?
Predicados altamente seletivos favorecem índices.

Tipo de Índice Disponível

Índice primário? Secundário? Hash? Cada tipo tem custos diferentes.

Custo Estimado de E/S

Comparação entre o custo de usar o índice versus varrer o arquivo inteiro.

 Percepção Importante

Índices **nem sempre são a melhor opção**. Para predicados com baixa seletividade (que retornam muitas tuplas), uma varredura sequencial pode ser mais eficiente do que múltiplos acessos aleatórios via índice.

Seleção com Igualdade: Índice sobre Chave

Predicados de igualdade sobre chave primária garantem que no máximo uma tupla será retornada, permitindo otimizações significativas.

Busca no Índice

O otimizador usa o índice (B+ ou hash) para localizar rapidamente a entrada correspondente ao valor buscado.

Acesso Direto

Um único acesso ao arquivo de dados recupera a tupla desejada a partir da entrada encontrada no índice.

Retorno do Resultado

A tupla é retornada sem verificações adicionais, graças à restrição de unicidade.

Custo de E/S

Para uma árvore B+ de altura h :

- $h + 1$ transferências de bloco no total
- h acessos para navegar no índice
- 1 acesso para recuperar a tupla

Eficiência

Um dos cenários mais eficientes em bancos de dados. Com índices bem dimensionados, h é tipicamente 3–4, mesmo para tabelas com milhões de registros.

Seleção com Igualdade: Índice Primário

Um índice primário sobre a chave de ordenação permite localizar eficientemente múltiplas tuplas que satisfazem um predicado de igualdade.

Localização Inicial

O índice primário localiza o primeiro registro que satisfaz o predicado de igualdade

Leitura Sequencial

Como o arquivo está ordenado pela chave, todos os registros correspondentes estão fisicamente adjacentes

Acesso Eficiente

Registros adicionais são lidos sequencialmente até que a condição não seja mais satisfeita

Custo: $h + b$ acessos a blocos, onde:

- h = altura da árvore do índice
- b = número de blocos com registros que satisfazem o predicado

Vantagem chave: Localidade física dos dados minimiza movimentos do cabeçote do disco, tornando índices primários ideais para predicados com múltiplos resultados.

Seleção com Igualdade: Índice Secundário

Índices secundários são construídos sobre atributos que não são a chave de ordenação, logo os registros correspondentes não estão fisicamente agrupados.

Busca no Índice	Lista de Ponteiros	Acessos Aleatórios
Navegação na estrutura do índice secundário (h acessos)	Recuperação da lista com n ponteiros para registros	Até n acessos ao disco para recuperar cada registro individualmente

Custo: Pior Caso

$h + n$ transferências de bloco

Onde n é o número de registros que satisfazem o predicado. Cada registro pode estar em um bloco diferente.

Custo: Caso Otimista

Se alguns registros compartilham blocos, o custo pode ser menor. Porém, essa otimização é difícil de prever.

📌 **Atenção:** Para predicados com baixa seletividade, um índice secundário pode ser **menos eficiente** que uma varredura completa devido aos acessos aleatórios.

Seleção com Predicado de Comparação

Predicados de comparação requerem estratégias distintas conforme o tipo de índice disponível.

Com Índice Ordenado

Árvores B+ permitem navegação eficiente. Para $A \geq v$, localiza-se v e percorre-se para frente; para $A \leq v$, percorre-se desde o início até v .

Sem Índice Adequado

Varredura completa do arquivo é necessária, examinando cada tupla para verificar se satisfaz a condição de comparação.

Custos por Tipo de Acesso

Índice Primário ($\geq$ ou $\leq$)

Custo: $h + b$ blocos, onde b é o número de blocos contendo resultados. Leitura sequencial após localização inicial.

Índice Secundário ($\geq$ ou $\leq$)

Custo: $h + n$ blocos (pior caso), onde n é o número de tuplas. Acessos potencialmente aleatórios a cada registro.

Varredura de Arquivo

Custo: br blocos, onde br é o tamanho total da relação. Exame de todas as tuplas.

Seleção com Predicado Composto: Conjunção

Quando múltiplas condições ($\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n$) são conectadas por AND, o otimizador pode explorar diferentes estratégias dependendo dos índices disponíveis.

Estratégia 1: Índice Único

Selecione um índice sobre o atributo de maior seletividade. Use-o para recuperar tuplas candidatas e avalie as demais condições em memória.

Estratégia 2: Índice Composto

Se existe um índice composto sobre múltiplos atributos do predicado, use-o para filtrar simultaneamente por várias condições.

Estratégia 3: Interseção de Índices

Use múltiplos índices separados, recupere os identificadores de registro de cada um e compute a interseção. Acesse apenas os registros resultantes.



Escolha da Estratégia

A estratégia 3 é vantajosa quando cada condição individual tem baixa seletividade, mas a conjunção é altamente seletiva. A estratégia 1 é preferível quando uma condição já é altamente seletiva.

Seleção com Predicado Composto: Disjunção

Predicados com disjunção ($\theta_1 \vee \theta_2 \vee \dots \vee \theta_n$) são mais desafiadores que conjunções, pois basta uma condição ser verdadeira para limitar as oportunidades de otimização.

Com Índices em Todas as Condições

Se há índices disponíveis para todos os atributos mencionados na disjunção:

- Use cada índice para recuperar conjuntos de identificadores
- Compute a **união** dos conjuntos de identificadores
- Acesse os registros correspondentes e elimine duplicatas

Custo com Índices Completos

Soma dos custos de uso de cada índice individual, mais o custo de unir os resultados e eliminar duplicatas.

Sem Índices Completos

Se falta índice para qualquer uma das condições na disjunção:

- Necessário realizar **varredura completa** do arquivo
- Avaliar o predicado completo para cada tupla
- Não é possível otimizar parcialmente

Razão: Mesmo que alguns atributos tenham índice, não podemos garantir que capturamos todas as tuplas sem examinar o arquivo inteiro.

Custo sem Índices Completos

br transferências de bloco para varrer toda a relação, independente de quantos índices parciais existam.

Ordenação e Avaliação de Consultas em Bancos de Dados

Técnicas de ordenação e operações de junção para processamento eficiente de consultas em SGBDs relacionais.

Estratégias de Ordenação para Relações

Relações que cabem na memória

Quando a relação cabe na memória principal, algoritmos como quicksort são aplicados diretamente, sem acessos adicionais ao disco.

- Acesso rápido aos dados
- Sem overhead de I/O
- Ideal para datasets pequenos

Relações que não cabem na memória

Quando a relação excede a memória disponível, utiliza-se o merge-sort externo, dividindo os dados em partições e ordenando em múltiplas passagens.

- Operações sequenciais em disco
- Divisão em partições gerenciáveis
- Processamento em múltiplas fases

Sort-Merge: Princípio Geral

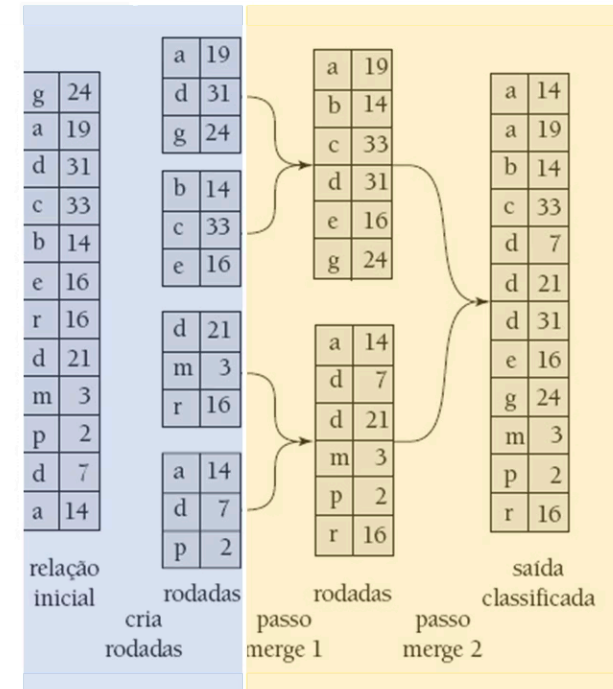
Fase 1: Ordenação Inicial

A primeira fase envolve a criação de runs ordenados. O algoritmo lê blocos de dados na memória, ordena-os internamente e grava os resultados como runs temporários no disco.

Fase 2: Mesclagem Ordenada

Na segunda fase, os runs ordenados são mesclados iterativamente em runs maiores, mantendo a ordem global. O processo continua até restar um único run ordenado com todos os dados.

1. Fase de ordenação



2. Fase de mesclagem ordenada

O diagrama ilustra o fluxo de dados pelas duas fases principais: particionamento e mesclagem progressiva até a ordenação completa.

Merge-Sort Externo

Ordenar relações **maiores que a memória** em duas fases:
criação de runs ordenados e mesclagem progressiva.

Custo Total

$$Custo = b_r \times (2 \times \lceil \log_{M-1}(b_r/M) \rceil + 1)$$

- b_r = número de blocos da relação
- M = buffers disponíveis em memória

ENTRADA: relação R com b_r blocos; M buffers de memória

PRÉ-CONDIÇÃO: $M \geq 2$

PASSO 1 — Criação de runs ordenados

$i \leftarrow 0$

enquanto R não foi completamente lida:

ler M blocos de R para a memória

ordenar os M blocos internamente

gravar run_i no disco

$i \leftarrow i + 1$

$N_runs \leftarrow i$

PASSO 2 — Mesclagem progressiva

enquanto $N_runs > 1$:

para cada grupo de $(M - 1)$ runs:

abrir $(M - 1)$ buffers de entrada, 1 buffer de saída

mesclar os runs usando heap mínimo sobre os ponteiros

gravar run mesclado no disco

$N_runs \leftarrow \lceil N_runs / (M - 1) \rceil$

SAÍDA: relação R completamente ordenada em disco

PÓS-CONDIÇÃO: número de passadas = $\lceil \log_{M-1}(b_r / M) \rceil + 1$

Complexidade do Merge-Sort Externo

O fator dominante no tempo de execução é o número de acessos a disco.

Fase de Ordenação

Cada bloco é lido uma vez e gravado uma vez, resultando em $2 \times br$ operações de I/O. Esta fase tem custo linear em relação ao tamanho dos dados.

Fase de Mesclagem

O número de passadas é $\lceil \log_{M-1}(br/M) \rceil$, com custo de $2 \times br$ por passada. Onde M é o número de buffers e br é o número de blocos da relação.

Custo Total

Somando as fases, teríamos $br \times (2\lceil \log_{M-1}(br/M) \rceil + 2)$. Como a escrita do resultado final não é contabilizada (ele pode ser enviado diretamente ao operador seguinte), subtrai-se br : $br \times (2\lceil \log_{M-1}(br/M) \rceil + 1)$ transferências de bloco.

Avaliação de Operações de Conjunto

Operações Fundamentais

As operações de conjunto são essenciais em consultas relacionais:

- **União ($\cup$):** Combina tuplas de duas relações
- **Interseção ($\cap$):** Retorna tuplas comuns
- **Diferença ($-$):** Remove tuplas de uma relação

Estratégias de Implementação

Duas abordagens principais são utilizadas:

Baseada em Ordenação

Ordena ambas as relações e processa sequencialmente

Baseada em Hashing

Usa funções hash para particionar e comparar eficientemente

❏ Ambas as abordagens requerem pré-processamento das relações, seja por ordenação ou por particionamento via função de hash.

Operação de Junção

A **junção** combina tuplas de duas relações com base em uma condição, sendo uma das operações mais custosas em bancos de dados relacionais.

Nested Loop Join

Abordagem iterativa que compara cada tupla de uma relação com todas as tuplas da outra. Eficiente para relações pequenas ou quando índices estão disponíveis.

Merge Join

Requer que ambas as relações estejam ordenadas pelo atributo de junção. Muito eficiente para grandes conjuntos ordenados por realizar uma varredura linear sincronizada.

Hash Join

Usa funções hash para particionar ambas as relações em buckets correspondentes. Excelente desempenho para grandes volumes quando há memória suficiente.

A escolha do algoritmo de junção pode impactar dramaticamente o desempenho de uma consulta.

Junção de Loop Aninhado em Bloco

O algoritmo processa a relação externa em **blocos de (M-2) páginas**, varrendo completamente a relação interna para cada bloco.

Custo

$$b_r + \left\lceil \frac{b_r}{M-2} \right\rceil \times b_s$$

Parâmetros

- **b_r** = blocos da relação externa (r)
- **b_s** = blocos da relação interna (s)
- **M** = buffers disponíveis

ENTRADA: relações **r** (externa) e **s** (interna); **M buffers**

PRÉ-CONDIÇÃO: $M \geq 3$ (**M-2** para **r**, **1** para **s**, **1** para saída)

PASSO 1 — Iterar sobre blocos da relação externa

para cada chunk de (**M - 2**) blocos de **r**:

carregar chunk de **r** na memória

PASSO 2 — Varrer relação interna

para cada bloco **b_s** de **s**:

carregar **b_s** na memória

para cada tupla **t_r** no chunk de **r**:

para cada tupla **t_s** em **b_s**:

se **t_r** e **t_s** satisfaz condição de junção:

adicionar (**t_r**, **t_s**) ao buffer de saída

se buffer de saída **cheio**: gravar no disco

SAÍDA: relação resultado da junção **r** e **s**

PÓS-CONDIÇÃO: cada bloco de **s** é lido $\lceil b_r / (M-2) \rceil$ vezes

Junção de Loop Aninhado em Bloco: Pior Caso

O pior cenário ocorre quando a memória é extremamente limitada, permitindo apenas um bloco de cada relação mais um bloco de saída.

Blocos Mínimos (3)

Apenas 3 blocos de memória disponíveis

Acessos Totais ($b_1 \times b_2$)

Cada bloco externo requer varredura completa da interna

Fórmula de Custo no Pior Caso:

$$Custo = b_r + (b_r \times b_s)$$

Para cada bloco da relação externa, toda a relação interna é lida do disco, resultando em um número quadrático de operações de I/O.

Isso ilustra a importância crítica da alocação adequada de memória para operações de junção em SGBDs.

Junção de Loop Aninhado em Bloco: Melhor Caso

O melhor cenário ocorre quando há memória suficiente para carregar completamente a menor relação.

Condição Ideal

Memória suficiente para toda a relação interna: $M \geq b_s + 2$

Varredura Única

Cada relação é lida exatamente uma vez do disco

Performance Máxima

Custo linear minimizado em relação ao tamanho dos dados

Fórmula de Custo no Melhor Caso:

$$Custo = b_r + b_s$$

Este é o cenário ideal onde:

- Toda a relação interna cabe na memória
- Apenas uma varredura da relação interna é necessária
- O número de operações de I/O é minimizado

Implicações práticas:

Otimizadores de consulta sempre tentam alocar a menor relação como interna e maximizar o uso de memória disponível para aproximar-se deste caso ideal.

Junção de Loop Aninhado em Bloco: Caso Geral

No caso geral, a memória permite carregar múltiplos blocos da relação externa, mas não toda a relação.

Fórmula de Custo no Caso Geral:

$$Custo = b_r + \left\lceil \frac{b_r}{M - 2} \right\rceil \times b_s$$

Variáveis da Fórmula

- **br:** Blocos da relação externa
- **bs:** Blocos da relação interna
- **M:** Blocos de memória disponíveis
- **M-2:** Blocos utilizáveis para a relação externa

Interpretação

O termo $\lceil br/(M-2) \rceil$ representa o número de vezes que a relação interna é varrida, processando (M-2) blocos externos por iteração.

Mais memória reduz drasticamente o custo: dobrar a memória pode reduzir o custo pela metade.

Exemplo de Custos de Junção de Loop Aninhado

Analizando a operação depositante ⋈ cliente, com depositante como relação **externa** (menor, 100 blocos) e cliente como relação **interna** (maior, 400 blocos).

Dados de Entrada

- Tuplas em cliente: 10.000
- Tuplas em depositante: 5.000
- Blocos de cliente: 400
- Blocos de depositante: 100

Configuração

Relação Externa: depositante (menor, 100 blocos)

Relação Interna: cliente (maior, 400 blocos)

Pior Caso

$br + br \times bs = 100 + 100 \times 400 = \mathbf{40.100 \text{ blocos}}$

Melhor Caso

$br + bs = 100 + 400 = \mathbf{500 \text{ blocos}}$

Caso Geral com M blocos de memória (br=100, bs=400):

$$Custo = 100 + \lceil \frac{100}{M - 2} \rceil \times 400$$

M = 5 blocos

$\lceil 100/3 \rceil \times 400 + 100 = \mathbf{13.700 \text{ blocos}}$

M = 12 blocos

$\lceil 100/10 \rceil \times 400 + 100 = \mathbf{4.100 \text{ blocos}}$

Impacto da Memória

De 5 para 12 blocos, o custo reduz em **70%**, demonstrando o impacto da alocação de recursos.

Junção de Merge (Merge Join)

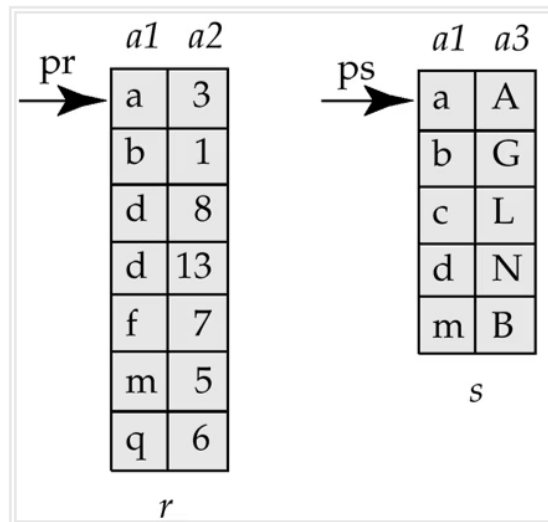
A **junção de merge** varre ambas as relações ordenadas simultaneamente, emitindo tuplas que satisfazem a condição de junção.

Custo (relações já ordenadas)

$$Custo = b_r + b_s$$

Parâmetros:

- **b_r** = número de blocos da relação r
- **b_s** = número de blocos da relação s
- Ambas devem estar ordenadas pelo atributo de junção
- Se não ordenadas, aplica-se merge-sort externo primeiro



ENTRADA: relações *r* e *s* ordenadas pelo atributo de junção A

PRÉ-CONDIÇÃO: *r* e *s* estão ordenadas por A (crescente)

PASSO 1 — Inicialização

p_r ← início de *r*; *p_s* ← início de *s*

PASSO 2 — Varredura sincronizada

enquanto *p_r* ≠ fim(*r*) e *p_s* ≠ fim(*s*):

se *r*[*p_r*].A < *s*[*p_s*].A:

avançar *p_r*

senão se *r*[*p_r*].A > *s*[*p_s*].A:

avançar *p_s*

senão: // *r*[*p_r*].A = *s*[*p_s*].A

PASSO 3 — Produto cartesiano local

marcar posição *p_{s_inicio}*

enquanto *s*[*p_s*].A = *r*[*p_r*].A:

emitir (*r*[*p_r*], *s*[*p_s*])

avançar *p_s*

avançar *p_r*

se *r*[*p_r*].A = *s*[*p_{s_inicio}*].A:

p_s ← *p_{s_inicio}* // reiniciar para próximo grupo

SAÍDA: relação resultado *r* *s*

PÓS-CONDIÇÃO: cada bloco lido no máximo uma vez

(sem duplicatas no atributo de junção)

Junção Merge: Análise Detalhada

Custo Total

$$Custo_{total} = Custo_{mesclagem} + Custo_{ordenação}$$

Custo de Ordenação

Se as relações não estiverem ordenadas:

- Relação r: $b_r \times (2\lceil \log_{M-1}(b_r/M) \rceil + 1)$
- Relação s: $b_s \times (2\lceil \log_{M-1}(b_s/M) \rceil + 1)$

Vantagens

- Performance previsível e linear
- Excelente para grandes volumes ordenados
- Aproveita índices existentes

Custo de Mesclagem

Caso típico: $b_r + b_s$ — cada bloco é lido exatamente uma vez.

Caso com Duplicatas

Muitos valores duplicados no atributo de junção podem exigir releitura de blocos, elevando o custo até $b_r \times b_s$ no pior caso.

Desvantagens

- Requer ordenação prévia (custo adicional)
- Pode ser custoso com muitas duplicatas
- Não ideal para dados não ordenados

Junção de Hash (Hash Join)

Ideia Central

Particionar ambas as relações com a mesma função hash **h** e, para cada par de partições, construir tabela hash da menor e sondar com a maior.

Custo

$$Custo = 3 \times (b_r + b_s)$$

Parâmetros

- **b_r, b_s** = blocos das relações r e s
- **n** = número de partições
- Requer $n \leq M - 1$
- Menor partição deve caber em memória

ENTRADA: relações **r** e **s**; função hash **h**; **M** buffers

PRÉ-CONDIÇÃO: $n \leq M - 1$; $\lceil b_s/n \rceil + 2 \leq M$

PASSO 1 — Fase de Particionamento (ambas as relações)

para cada tupla **t_r** em r:

$i \leftarrow h(t_r.A) \bmod n$

gravar **t_r** na partição **r_i**

para cada tupla **t_s** em s:

$i \leftarrow h(t_s.A) \bmod n$

gravar **t_s** na partição **s_i**

PASSO 2 — Fase de Sondagem (por partição)

para $i \leftarrow 0$ até $n - 1$:

carregar partição **s_i** na memória

construir tabela hash **H_i** usando $h'(t.A)$

para cada tupla **t_r** em **r_i**:

buscar **t_r.A** em **H_i**

para cada **t_s** correspondente em **H_i**:

se **t_r** **t_s** satisfaz condição de junção:

emitir (**t_r**, **t_s**)

SAÍDA: relação resultado **r** **s**

PÓS-CONDIÇÃO: cada bloco lido/gravado **3** vezes no total

Princípio da Junção de Hash

O diagrama ilustra o fluxo completo do algoritmo de hash join, mostrando como as relações são particionadas e processadas em pares.

Particionamento Inicial

Relação r dividida em n partições:

$r_0, r_1, \dots, r_{n-1}$

Particionamento Correspondente

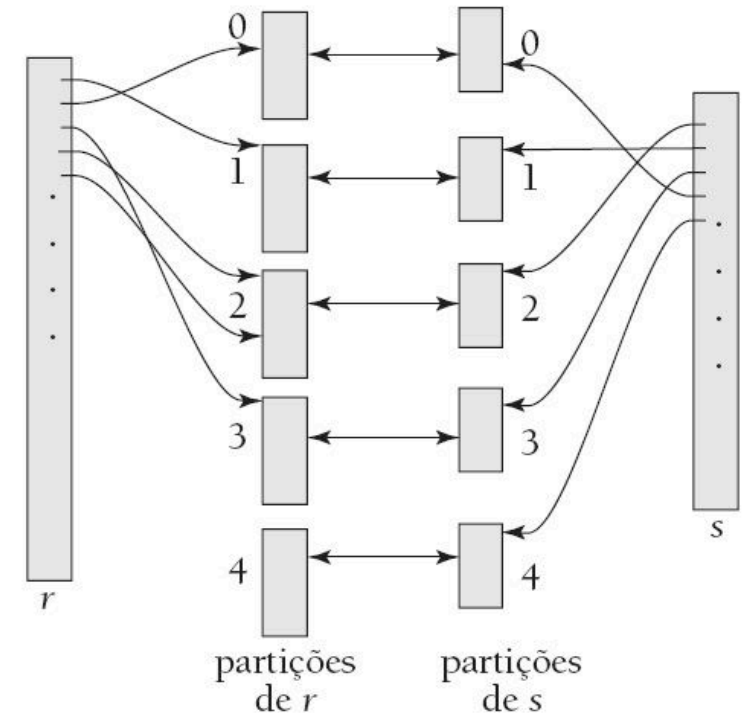
Relação s dividida nas mesmas n partições:

$s_0, s_1, \dots, s_{n-1}$

Processamento por Pares

Junção de r_i com s_i para todo i , combinando apenas partições correspondentes

Garantia de Correção: Tuplas que satisfazem a condição de junção sempre terão o mesmo valor hash, portanto estarão na mesma partição. Isso garante que nenhuma tupla de resultado seja perdida.



- ❏ O número de partições n é escolhido de forma que cada partição da relação de construção (s) caiba na memória: $n \geq \lceil b_s / M \rceil$. As partições da relação de sondagem (r) não precisam caber na memória.

Comparações via Junção de Hash

O número de comparações é influenciado pela qualidade da função hash e pela distribuição dos dados.

Caso Ideal

Distribuição uniforme perfeita: Cada partição tem aproximadamente o mesmo tamanho. Número de comparações $\approx (nr \times ns)/n$, onde n é o número de partições.

Caso Típico

Distribuição razoável: Pequenas variações no tamanho das partições. Número de comparações próximo ao ideal com overhead mínimo para tratar desbalanceamento.

Caso Problemático

Distribuição enviesada: Algumas partições muito grandes (skew). Pode degenerar para comparações próximas a $nr \times ns$ se uma partição contém a maioria das tuplas.

Otimizações Práticas

- Uso de funções hash múltiplas
- Re-particionamento de partições grandes
- Amostragem para detectar skew

Métricas de Qualidade

SGBDs modernos monitoram estatísticas de hash join:

- Fator de desbalanceamento de partições
- Taxa de colisões de hash
- Número de re-particionamentos necessários

Custo da Junção de Hash e Comparação com Merge Join

Consolidando os custos dos principais algoritmos de junção e seus trade-offs de desempenho.

Fases do Hash Join Inclui particionamento, construção da tabela hash e sondagem.	Varreduras por Relação Cada relação é lida duas vezes durante o processo.	Número de Partições Escolhido como $n \geq \lceil bs/M \rceil$, onde s é a relação de construção (build), cujas partições precisam caber na memória.
--	---	---

Fórmula de Custo do Hash Join:

$$Custo_{hash} \approx 3(b_r + b_s)$$

Hash Join Melhor para: Grandes volumes de dados não ordenados com memória suficiente. Custo típico: $3(br + bs)$	Merge Join Melhor para: Dados já ordenados ou quando a saída ordenada é necessária. Custo típico: $br + bs$ (se já ordenado)	Nested Loop Melhor para: Relações pequenas ou com índices disponíveis na relação interna. Custo típico: $br + (br \times bs)/(M-2)$
---	---	--

❏ A escolha ideal depende do tamanho das relações, memória disponível, índices e ordenação prévia.

Avaliação de Expressões em Bancos de Dados

Uma consulta SQL é convertida em uma árvore de expressões hierárquica que o banco de dados avalia usando diferentes estratégias para recuperar e transformar dados com eficiência.

Estratégias de Avaliação

Materialização

Gera e armazena **resultados intermediários** em disco. Cada operação é executada completamente antes da próxima, com resultados persistidos em relações temporárias.

Pipelining

Passa tuplas adiante para operações ancestrais **durante a execução**, sem esperar a conclusão completa de cada etapa. Funciona como uma linha de produção contínua.

Materialização: Avaliação Passo a Passo

A **avaliação materializada** processa uma operação por vez, começando pelo nível mais baixo da árvore de expressões e subindo até completar toda a consulta.

Identificar operação base	Executar e materializar	Subir na hierarquia	Repetir até o topo
Localiza a operação mais básica na árvore de expressões - aquela que não depende de nenhum resultado intermediário	Executa completamente a operação e armazena o resultado em uma relação temporária no disco	Utiliza os resultados materializados como entrada para as operações do nível superior	Continua o processo até que todas as operações sejam executadas e o resultado final seja obtido

Exemplo Prático

Considere uma consulta que precisa:

1. Filtrar contas com saldo < 2500
2. Fazer junção com a tabela cliente
3. Projetar apenas nome_cliente

Na materialização, o sistema:

1. **Calcula e armazena** osaldo<2500(conta)
2. **Lê do disco** e faz junção com cliente
3. **Projeta** o resultado final

Aspectos Críticos da Materialização

✓ Aplicabilidade Universal

A avaliação materializada **sempre funciona**, independentemente da complexidade da consulta ou dos tipos de operações envolvidas. É a abordagem mais robusta e confiável.

✗ Alto Custo de I/O

O maior problema é o **custo elevado de escrita e leitura** de resultados intermediários. Cada operação gera disco I/O adicional, que pode dominar o tempo total de execução.

Fórmula de Custo Total

$$\text{Custo Total} = \sum_{i=1}^n \text{Custo}_{\text{operação}_i} + \text{Custo}_{\text{I/O intermediário}}$$

Otimização: Buffer Duplo

O **buffer duplo** utiliza dois buffers de saída por operação — um sendo escrito no disco (**Buffer A**) e outro sendo preenchido com novos dados (**Buffer B**). Isso permite sobrepor escrita em disco e computação, reduzindo o tempo total de execução da consulta.

Pipelining: Processamento Contínuo

A **avaliação em pipeline** permite que múltiplas operações executem simultaneamente, passando resultados parciais entre si de forma contínua.

Filtrar

Processa tuplas e passa resultados imediatamente

Junção

Recebe tuplas filtradas e gera junções incrementalmente

Projetar

Aplica projeção conforme tuplas chegam

Vantagens e Limitações

Benefícios do Pipelining

- **Custo computacional reduzido** - não há necessidade de armazenar relações temporárias
- **Menor latência** - resultados começam a aparecer mais cedo
- **Uso eficiente de memória** - apenas dados ativos ocupam RAM

Quando Não é Possível

- **Operações de ordenação** - precisam ver todos os dados antes de gerar saída
- **Junção hash** - requer construção completa da tabela hash
- **Agregações complexas** - podem precisar de dados completos



Requisito chave: Os algoritmos devem gerar tuplas de saída à medida que recebem entradas, sem acumular todos os dados primeiro.

Pipelining Controlado por Demanda

Ideia Central

Cada operador implementa a **interface iterador** (open / next / close), e a operação raiz **puxa tuplas sob demanda**, eliminando a materialização de relações intermediárias.

Os Três Métodos

open()

Inicializa a operação e prepara estruturas

next()

Retorna a próxima tupla ou $\emptyset$ quando esgotado

close()

Libera recursos e finaliza a execução

ENTRADA: árvore de operadores O com raiz R; relações **base** nas folhas

PRÉ-CONDIÇÃO: cada operador implementa {open, next, close}

PASSO 1 — Inicialização (top-down)

```
R.open()
// propaga recursivamente para filhos:
// filho_esq.open(); filho_dir.open(); ...
```

PASSO 2 — Produção de **tuplas** (pull loop)

enquanto verdadeiro:

```
t ← R.next()
se t =  $\emptyset$ : interromper // pipeline esgotado
entregar t ao cliente
```

// Internamente, next() de cada operador:

```
// chama next() nos filhos conforme necessário,
// processa as tuplas recebidas,
// retorna uma tupla de saída ou  $\emptyset$ 
```

PASSO 3 — Finalização (top-down)

```
R.close()
// propaga recursivamente para filhos
```

SAÍDA: sequência de tuplas resultado da consulta

PÓS-CONDIÇÃO: nenhuma relação intermediária gravada em **disco**
(exceto operadores bloqueantes: sort, hash **join**)

Pipelining Controlado por Produtor

No modelo **push-based**, os operadores **produzem tuplas ativamente** e as empurram para seus pais sem esperar por solicitações.

Produção Ativa

Operadores filhos geram tuplas rapidamente e as colocam em buffers compartilhados

Controle de Fluxo

Se buffer enche, produtor aguarda; se esvazia, consumidor aguarda

Comparação: Demanda vs. Produtor

Controlado por Demanda

- Controle top-down (puxar)
- Processa sob solicitação
- Mais fácil de implementar

Desafios do Modelo Produtor

O sistema precisa de um **escalonador sofisticado** que monitore os buffers e decida quais operações executar a cada momento, garantindo que o pipeline flua sem bloqueios.

Buffer Intermediário

Mantido entre operadores - filho escreve, pai consome conforme capacidade

Escalonamento

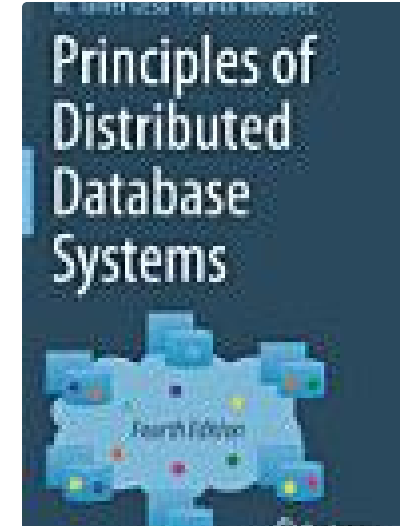
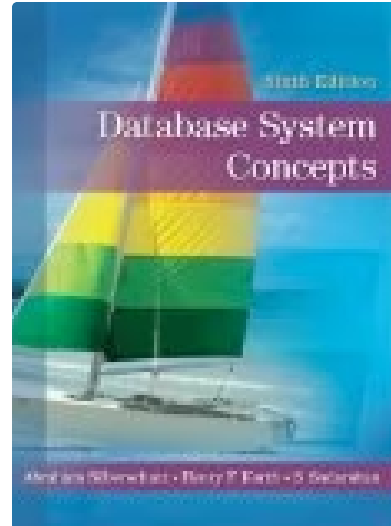
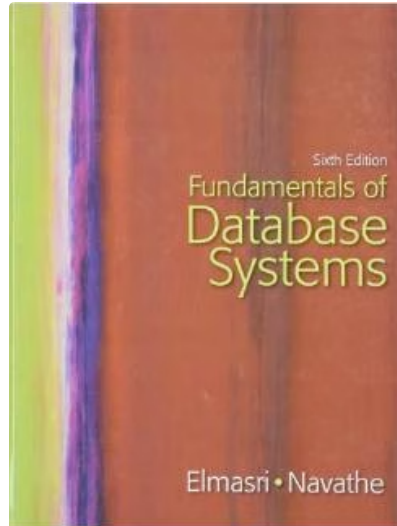
Sistema coordena operações com espaço disponível para processar

Controlado por Produtor

- Controle bottom-up (empurrar)
- Melhor paralelismo potencial
- Requer gestão cuidadosa de buffers

📌 **Importante:** Ambos os modelos requerem algoritmos que operem incrementalmente, gerando resultados parciais conforme processam os dados.

Referências



Elmasri & Navathe

Fundamentals of Database Systems

Pearson, 2016

Korth, Sudarshan & Silberschatz

Database System Concepts

McGraw-Hill, 2019

Özsu & Valduriez

Principles of Distributed Database Systems

Springer Nature, 2019